Adaptive CRDT Update Batching: Mathematical Formulation

26 October 2026

Contents

1 Problem statement **1**

2 System model and notation **1**

3 Workload and network model **2**

3.1 Edit arrivals 2

3.2 Edit content 3

3.3 Network 3

4 Decision variables and batching dynamics **3**

4.1 Flush rule 3

4.2 Link queue 3

4.3 Per-edit delay 4

5 Cost functionals **4**

6 The optimisation problem **4**

6.1 Main formulation (P1) 4

6.2 Robust variant (P2) 5

6.3 Bi-objective variant (P3) 5

6.4 Penalised form for the genetic algorithm 5

7 Analytic fluid model **5**

7.1 Assumptions 5

7.2 Result 1: renewal cycle 6

7.3 Result 2: byte rate 6

7.4 Result 3: delay 6

7.5 Result 4: optimal window 6

7.6 Worked example (illustrative numbers, to be replaced by measured values) 7

7.7 What the fluid model tells you . . . 7

8 Solution methods **7**

8.1 Why derivative-free search . . . 7

8.2 Methods . . . 8

8.3 Comparison protocol . . . 8

8.4 Validation against theory . . . 8

9 Extension: rate-adaptive policy **8**

10 Assumptions, validation and limitations **9**

10.1 Modelling assumptions . . . 9

10.2 Correctness checks . . . 9

10.3 Parameters to fix before final experiments . . . 9

10.4 Known limitations . . . 10

1 Problem statement

We choose a batching rule for CRDT updates that minimises network cost while keeping remote edits visible within a latency limit, and without ever breaking replica convergence.

In a collaborative editor, N users edit one shared document, each on a local replica. A CRDT (for example Yjs) guarantees that all replicas converge to the same state once every update has been delivered, whatever the delivery order. It does not decide how updates travel. Each message carries a fixed header and framing cost regardless of how little it contains, so sending every keystroke as its own message wastes bytes.

Buffering edits and sending them together amortises that fixed cost, because adjacent edits also merge into a more compact encoding. But every buffered edit waits, so remote users see changes later. The trade-off is therefore between **bytes on the wire** and **time until a remote replica applies the edit**.

A batching policy has two knobs: a maximum wait time T and a maximum batch size S. The questions are:

1. For a given workload and network, which (T, S) minimises a weighted combination of bytes and tail latency, subject to a latency service-level objective (SLO)?
2. Can this optimum be characterised analytically, and when is it necessary to search for it numerically?
3. Does a policy that adapts to the current edit rate beat the best fixed policy?

The CRDT itself is treated as a fixed component. Eventual convergence is a correctness invariant that every policy must preserve, not a quantity to optimise.

2 System model and notation

The system is N replicas that generate edits, buffer them under a batching policy, and send them over one bottleneck link to all other replicas. All quantities below are fixed before the experiments

and reported in the final write-up.

Symbol	Meaning	Unit
N	number of replicas (users)	count
Θ	observation horizon	ms
$t_j,\ k_j,\ s_j$	time, author and encoded size of edit j	ms, index, bytes
$\lambda_k,\ \lambda$	edit rate of user k; rate of a single stream	edits/ms
H	per-message header and framing overhead	bytes
$s_0,\ s_1$	per-run overhead and per-edit payload of the CRDT encoding	bytes
p_c	probability an edit extends the previous contiguous run	–
$\kappa,\ \tilde{s}$	fixed bytes per batch; effective bytes per edit (Section 7)	bytes
R	link rate	bytes/ms
d	one-way propagation delay	ms
φ	fan-out factor ($N-1$ peer-to-peer, or $1+(N-1)$ via a relay)	–
T	batching window (maximum wait)	ms
S	batch size cap	bytes
$x=(T,S)$	batching policy	–
$a_b,\ f_b$	opening and flush time of batch b	ms
$m_b,\ r_b,\ P_b$	edits, contiguous runs and payload bytes of batch b	count, count, bytes
$\sigma_b,\ W_b$	service time and queueing wait of batch b	ms
D_j	end-to-end delay of edit j	ms
q	tail quantile (0.95)	–
L_{SLO}	latency service-level objective	ms
$\mathcal{B}_0$	bytes of the baseline (immediate send) on the same trace	bytes
$w_b,\ w_l$	objective weights, $w_b+w_l=1$	–
$\omega,\ \omega_i,\ K$	scenario (trace + network); i-th of K training scenarios	–

3 Workload and network model

A scenario ω is one complete, reproducible trace plus one network configuration. Every policy is evaluated on the same set of scenarios, using common random numbers.

3.1 Edit arrivals

Edits of user k form a point process $\{t_j : k_j = k\}$ with intensity $\lambda_k(t)$. Three models of increasing realism:

1. **Poisson:** $\lambda_k(t) = \lambda_k$ constant. This is the tractable baseline used for the analytic model in Section 7.
2. **Two-state Markov-modulated Poisson (MMPP):** the rate switches between λ_{lo} (idle, reading) and λ_{hi} (typing burst), with exponential sojourn times of means $1/\nu_{\mathrm{lo}}$ and $1/\nu_{\mathrm{hi}}$. This captures bursty typing.

3. **Hawkes:** $\lambda_k(t) = \mu_k + \sum_{t_i<t} \alpha\, e^{-\beta(t-t_i)}$, a self-exciting process in which each edit raises the short-term chance of the next.

3.2 Edit content

Each edit j has an encoded size s_j. Whether it extends the previous run (the same author typing at an adjacent position) is a Bernoulli(p_c) event. Contiguous runs matter because the CRDT merges them into one struct, so a batch of m edits has an expected number of runs

$$\mathbb{E}[r_b \mid m_b = m] = 1 + (1 - p_c)(m - 1) = p_c + (1 - p_c)\, m. \tag{1}$$

The constants s_0, s_1 and p_c are estimated by encoding real edit traces with the chosen CRDT library, not assumed.

3.3 Network

A single bottleneck link of rate R with FIFO service, fixed propagation delay d and per-message header H. Delivery is reliable, so each message eventually arrives. Scenario families: light (2–5 users, sporadic), normal (5–10, continuous typing), burst (10–20, synchronised bursts), and constrained network (high d, low R).

4 Decision variables and batching dynamics

The decision vector is $x = (T, S)$ in the box $\mathcal{X} = [T_{\min}, T_{\max}] \times [S_{\min}, S_{\text{cap}}]$. Setting $T = 0$ recovers immediate transmission, which serves as the baseline. A per-user extension $x = (T_k, S_k)_{k=1}^{N}$ has dimension $2N$.

4.1 Flush rule

Batch b opens when the first edit arrives into an empty buffer, at time a_b. It flushes at the earlier of the timer expiring and the size cap being reached:

$$f_b = \min\Big(a_b + T,\ \inf\Big\{t \ge a_b : \sum_{j:\, a_b \le t_j \le t} s_j \ \ge S\Big\}\Big). \tag{2}$$

The batch contains the edits $\mathcal{B}_b = \{j : a_b \le t_j \le f_b\}$, with $m_b = |\mathcal{B}_b|$. Its payload is

$$P_b = s_0\, r_b + s_1\, m_b. \tag{3}$$

In the experiments P_b is replaced by the measured length of the merged CRDT update (for Yjs, the output of `mergeUpdates` over the batch), and the formula above is used only for analysis.

4.2 Link queue

The service time of batch b is $\sigma_b = (H + P_b)/R$. Waiting time at the link follows the Lindley recursion, with batches indexed in flush order:

$$W_{b+1} = \max\big(0,\ W_b + \sigma_b - (f_{b+1} - f_b)\big), \qquad W_1 = 0. \tag{4}$$

4.3 Per-edit delay

An edit j that belongs to batch b waits for the flush, then for the queue, then for transmission and propagation:

$$D_j = \underbrace{(f_b - t_j)}_{\text{batching}} + \underbrace{W_b}_{\text{queue}} + \underbrace{\sigma_b}_{\text{transmit}} + \underbrace{d}_{\text{propagate}} . \tag{5}$$

Batching lowers the flush rate, which shrinks the queueing term. The batching term grows with T. The optimisation exists because these effects pull in different directions.

5 Cost functionals

For a policy x and scenario ω, three quantities are computed from one simulated run. Bytes and tail latency enter the objective; message count and utilisation are diagnostics and constraints.

$$\mathcal{B}(x;\omega) = \varphi \sum_b (H + P_b), \tag{6}$$

$$L_q(x;\omega) = \inf\Big\{\ell : \ \tfrac{1}{n}\,\#\{j : D_j \le \ell\} \ \ge q\Big\}, \qquad q = 0.95, \tag{7}$$

$$\rho(x;\omega) = \frac{\sum_b \sigma_b}{\Theta}, \qquad \mathcal{M}(x;\omega) = \#\{b\}. \tag{8}$$

Here n is the number of edits, ρ is link utilisation (stability requires $\rho < 1$) and $\mathcal{M}$ is the message count.

Both cost terms are normalised so that the weights are comparable:

- Bytes are divided by $\mathcal{B}_0(\omega)$, the bytes of immediate transmission ($T = 0$) on the same trace.
- Latency is divided by the SLO L_{SLO}.

This makes the byte term equal 1 for the baseline and the latency term equal 1 exactly at the SLO.

Because L_q is a quantile of a sample of delays, it is piecewise constant in x and non-differentiable. The flush rule contains a min and an inf. So $(\mathcal{B}, L_q)$ are not smooth functions of x, which is why gradient-based methods are not applied directly to the simulated objective.

6 The optimisation problem

The main formulation is a constrained stochastic program, approximated by averaging over K training scenarios (sample-average approximation).

6.1 Main formulation (P1)

$$\min_{x \in \mathcal{X}} \ J(x) = \frac{1}{K} \sum_{i=1}^{K} \left[w_b \, \frac{\mathcal{B}(x;\omega_i)}{\mathcal{B}_0(\omega_i)} + w_l \, \frac{L_{0.95}(x;\omega_i)}{L_{\text{SLO}}} \right] \tag{9}$$

subject to

$$g_1(x) = \frac{1}{K} \sum_{i=1}^{K} L_{0.95}(x;\omega_i) - L_{\text{SLO}} \ \le 0, \tag{10}$$

$$g_2(x) = \rho(x;\omega_i) - \rho_{\max} \ \le 0 \quad \forall i, \qquad \rho_{\max} < 1, \tag{11}$$

$$\mathcal{X} = [T_{\min}, T_{\max}] \times [S_{\min}, S_{\text{cap}}], \tag{12}$$

and the **hard invariant** that for every $x \in \mathcal{X}$ and every ω_i, all replicas hold identical document state after all updates are delivered. This holds for any policy under reliable delivery, because CRDT merges commute. It is checked empirically in every run, not enforced as an algebraic constraint. A policy that violates g_1, g_2 or the invariant is infeasible and is never reported as an optimum.

6.2 Robust variant (P2)

The average can hide bad workloads. Minimise the conditional value at risk of the per-scenario objective $J_i(x)$ at level α:

$$\min_{x \in \mathcal{X},\ \eta \in \mathbb{R}} \ \eta + \frac{1}{(1-\alpha)K} \sum_{i=1}^{K} \big[J_i(x) - \eta\big]^+. \tag{13}$$

6.3 Bi-objective variant (P3)

Without fixing weights, treat bytes and latency as competing objectives:

$$\min_{x \in \mathcal{X}} \ \big(\bar{\mathcal{B}}(x),\ \bar{L}_{0.95}(x)\big). \tag{14}$$

The Pareto front is traced by the ε-constraint method: minimise $\bar{\mathcal{B}}(x)$ subject to $\bar{L}_{0.95}(x) \le \varepsilon$ and sweep ε over a grid up to L_{SLO}. A weighted sum only finds Pareto points on the convex hull of the achievable set, so the ε-constraint sweep is the safer way to draw the full front.

6.4 Penalised form for the genetic algorithm

The GA minimises an unconstrained fitness with a quadratic penalty for the latency constraint, where μ is large relative to J:

$$\tilde{J}(x) = J(x) + \mu\big([g_1(x)]^+\big)^2. \tag{15}$$

7 Analytic fluid model

Under simplifying assumptions the problem has a closed-form solution in which the latency SLO is always active and the best window follows a square-root law in the edit rate. This gives the simulator and the optimisers a known answer to be checked against.

7.1 Assumptions

1. One sender with Poisson arrivals of rate λ, and a timer-only policy ($S = \infty$).
2. Negligible queueing ($W_b \approx 0$, so ρ is small).
3. Constant transmission term $c_{\mathrm{net}} = d + (H + \bar{P})/R$.
4. Edits are generated by the process of Section 3 with run-continuation probability p_c.

With $\mathbb{E}[r_b \mid m] = p_c + (1 - p_c)m$ from Section 3, a batch of m edits has expected size

$$H + P_b = \underbrace{(H + p_c s_0)}_{\kappa} + \underbrace{\big(s_1 + (1 - p_c)s_0\big)}_{\tilde{s}}\, m_b, \tag{16}$$

so κ is the fixed cost per batch and $\tilde{s}$ is the marginal cost per edit.

7.2 Result 1: renewal cycle

A batch opens at the first arrival, then collects arrivals for time T. By memorylessness the next batch opens after an $\mathrm{Exp}(\lambda)$ idle gap, so one cycle has mean length $T + 1/\lambda$. The number of other edits in the window is $\mathrm{Poisson}(\lambda T)$, so

$$\mathbb{E}[m] = 1 + \lambda T, \qquad \text{message rate} = \frac{\lambda}{1 + \lambda T}. \tag{17}$$

7.3 Result 2: byte rate

Multiplying message rate by expected bytes per batch gives

$$\beta(T) = \varphi\Big(\lambda \tilde{s} + \frac{\lambda \kappa}{1 + \lambda T}\Big). \tag{18}$$

The immediate-send baseline is $\beta(0) = \varphi\lambda(\kappa + \tilde{s})$. So the ratio to baseline is

$$\frac{\beta(T)}{\beta(0)} = \frac{\tilde{s} + \kappa/(1 + \lambda T)}{\kappa + \tilde{s}}, \tag{19}$$

which is decreasing and convex in T, and tends to $\tilde{s}/(\kappa + \tilde{s})$ as $T \to \infty$. **Batching can therefore never save more than the fraction** $\kappa/(\kappa + \tilde{s})$, the share of bytes that is fixed per-message overhead. If that share is small, batching is not worth its latency cost.

7.4 Result 3: delay

The first edit in a batch waits the full T. Each of the other $\mathrm{Poisson}(\lambda T)$ edits is uniform in the window, so it waits $T - U$ with U uniform on $[0, T]$, a mean of $T/2$. By renewal-reward, the mean batching delay per edit is

$$\mathbb{E}[D_{\text{batch}}] = \frac{T + \lambda T^2/2}{1 + \lambda T} = \frac{T\,(1 + \lambda T/2)}{1 + \lambda T}. \tag{20}$$

For the tail: the fraction of edits with batching delay above $w < T$ is $[1 + \lambda(T - w)]/(1 + \lambda T)$. Setting this to 0.05 gives

$$L_{0.95}(T) = \min\Big(T,\ 0.95\big(T + \tfrac{1}{\lambda}\big)\Big) + c_{\text{net}}. \tag{21}$$

The minimum equals T exactly when $\lambda T < 19$. That is the regime of interest for typical windows: the p95 delay is just the window plus the network term, and its slope in T is 1.

7.5 Result 4: optimal window

In the regime $\lambda T < 19$, take the weighted objective with normalised terms, $J(T) = w_b\,\beta(T)/\beta(0) + w_l\,L_{0.95}(T)/L_{\text{SLO}}$. Its derivative is

$$\frac{dJ}{dT} = -\frac{w_b\,\lambda\kappa}{(\kappa + \tilde{s})(1 + \lambda T)^2} + \frac{w_l}{L_{\text{SLO}}}. \tag{22}$$

J is convex (a convex decreasing byte term plus a linear latency term), so the stationary point is the global minimiser. Setting $dJ/dT = 0$ gives the **square-root law**

$$T^*_{\text{unc}} = \frac{1}{\lambda}\left[\sqrt{\frac{w_b}{w_l} \cdot \frac{\lambda\,\kappa\,L_{\text{SLO}}}{\kappa + \tilde{s}}} - 1\right]^+. \tag{23}$$

The SLO g_1 is linear in T with slope 1, so it caps the window at $T_{\text{SLO}} = L_{\text{SLO}} - c_{\text{net}}$. The optimum of the constrained problem is

$$T^* = \min\left(T^*_{\text{unc}},\ T_{\text{SLO}},\ T_{\max}\right). \tag{24}$$

If the size cap is active instead, replace T by the time to fill it, $T_S = (S/\tilde{s} - 1)/\lambda$, and the effective window is $\min(T, T_S)$.

7.6 Worked example (illustrative numbers, to be replaced by measured values)

Let $\lambda = 0.01$ edits/ms (10 per second), $\kappa = 100$ bytes, $\tilde{s} = 20$ bytes, $d = 30$ ms with negligible transmission time, $L_{\text{SLO}} = 100$ ms, $w_b = 0.8$, $w_l = 0.2$.

- Square-root law: $\sqrt{4 \times 0.01 \times 100 \times 100/120} = 1.826$, so $T^*_{\text{unc}} = (1.826 - 1)/0.01 \approx 83$ ms.
- SLO cap: $T_{\text{SLO}} = 100 - 30 = 70$ ms, which binds. So $T^* = 70$ ms.
- At $T = 70$ ms, $\lambda T = 0.7$ and the byte ratio is $(20 + 100/1.7)/120 \approx 0.66$, so about 34% fewer bytes than immediate sending.
- The limiting saving is $\kappa/(\kappa + \tilde{s}) = 100/120 \approx 83\%$, which would need unbounded delay.
- With equal weights ($w_b = w_l$) the square root is $0.91 < 1$, so $T^*_{\text{unc}} = 0$: batching is not worth it at this rate and SLO.

7.7 What the fluid model tells you

- Heavier edit rate λ and larger fixed overhead κ favour longer windows.
- A tight SLO or a heavy weight on latency pushes T^* to zero.
- The optimum is on the SLO boundary whenever the unconstrained optimum would violate it, so the constraint is **active**, not just present.
- Everything beyond these cases (bursty arrivals, queueing, many users, merging variation) needs the simulator and a numerical optimiser.

8 Solution methods

The simulated objective is a noisy, non-smooth black box, so we compare derivative-free methods against a reference and against the analytic optimum of Section 7.

8.1 Why derivative-free search

1. Each evaluation of $J(x)$ is a stochastic simulation over K scenarios, so it carries sampling noise.
2. The p95 quantile and the flush rule make J piecewise constant or non-differentiable in places.
3. Bursty arrivals, the queueing term and variable run merging remove the closed form.
4. In the per-user extension, x has dimension $2N$, so an exhaustive grid with G points per axis needs G^{2N} evaluations, which is infeasible. This is where a metaheuristic has a real justification.

8.2 Methods

Method	Role	Notes
Immediate send ($T = 0$)	Responsiveness baseline	Defines $\mathcal{B}_0$
Fixed-window batching	Naive baseline	One hand-picked T, S for all scenarios
Fluid optimum T^*	Analytic benchmark	Exact only in the Poisson, no-queue case
Grid search	Reference optimiser	$G_1 \times G_2$ points over $\mathcal{X}$; exhaustive in 2D
Random search	Budget-matched comparator	n uniform samples from $\mathcal{X}$
Genetic algorithm	Metaheuristic	Two integer genes; population 30–50; tournament selection; uniform crossover; bounded mutation; elitism of 1–2; fitness $\tilde{J}$

8.3 Comparison protocol

- **Common random numbers:** every method sees the same K scenarios and seeds, so differences are not noise.
- **Equal evaluation budget E:** report the best feasible J after e evaluations for e up to E, for every method.
- **Replication:** run the GA and random search with 10 independent seeds; report mean $\pm$ standard deviation, and use a paired non-parametric test (Wilcoxon signed-rank) across scenarios.
- **Regret:** measure $J(\hat{x}) - J(x_{\text{grid}})$, with the grid solution as the reference in 2D.
- **Generalisation:** tune on K_{train} scenarios, then report J on held-out K_{test} scenarios. The gap $J_{\text{test}} - J_{\text{train}}$ measures overfitting.

8.4 Validation against theory

In the Poisson, light-load, timer-only setting, the simulator's measured byte rate, mean delay and p95 should match $\beta(T)$, $\mathbb{E}[D_{\text{batch}}]$ and $L_{0.95}(T)$ from Section 7 within Monte Carlo confidence intervals, and the grid optimum should land near T^*. If it does not, the simulator or the model has an error to fix before any optimiser result is trusted. A genuine finding is also acceptable here: if grid search matches or beats the GA in 2D, that is reported as a result, because the search space is small.

9 Extension: rate-adaptive policy

The square-root law implies the best window depends on the current edit rate, so a policy that tracks the rate should beat any single fixed window under bursty load. This is the hypothesis the extension tests.

The fluid optimum can be rewritten as $T^*(\lambda) = \sqrt{c/\lambda} - 1/\lambda$, where $c = w_b\,\kappa\,L_{\text{SLO}}/\big(w_l(\kappa + \tilde{s})\big)$. This motivates a two-parameter family with the window as a function of an estimated rate $\hat{\lambda}$:

$$T(\hat{\lambda}; \theta) = \text{clip}\left(\frac{a}{\sqrt{\hat{\lambda}}} - \frac{b}{\hat{\lambda}},\ T_{\min},\ \min(T_{\max}, T_{\text{SLO}})\right). \tag{25}$$

The rate estimate is an exponentially weighted moving average over windows of length Δ, where n_t is the number of edits observed in window t:

$$\hat{\lambda}_{t+1} = (1-\gamma)\,\hat{\lambda}_t + \gamma\,\frac{n_t}{\Delta}. \tag{26}$$

The policy parameters are $\theta = (a, b, \gamma)$. The fluid model supplies a principled starting point, $a = \sqrt{c}$ and $b = 1$, so the search is not random. The optimisation problem is

$$\min_{\theta}\ \frac{1}{K}\sum_{i=1}^{K} J\big(\pi_\theta; \omega_i\big) \quad \text{s.t. } g_1,\ g_2 \text{ as in Section 6} \tag{27}$$

with scenarios ω_i drawn from the MMPP and Hawkes families so that the rate genuinely varies. This is three-dimensional, which is small enough for a GA but already awkward for a fine grid. Compare it with the best fixed x^* on the same held-out scenarios. If the adaptive policy does not win, the result is reported as such.

10 Assumptions, validation and limitations

The model is only as good as its assumptions, and the report should state them rather than hide them.

10.1 Modelling assumptions

- Delivery is reliable and FIFO, with no message loss or reordering at the link.
- A single bottleneck link and a constant propagation delay d. No multi-hop or time-varying networks in the base model.
- Edit sizes and run-continuation probability p_c are stationary and estimated from real traces.
- Only document-edit updates are modelled. Cursor, presence and awareness traffic, and tombstone garbage collection, are out of scope.
- The latency SLO is measured end to end at the 95th percentile, not as a worst case.
- The fluid model additionally assumes Poisson arrivals, one sender and negligible queueing, and is used only as a benchmark.

10.2 Correctness checks

1. After every simulated run, quiesce (deliver all pending messages) and assert that all replicas have identical document state.
2. Run the same trace through the real CRDT library to confirm that merged update sizes match the P_b used by the simulator within a stated tolerance.
3. Test with adversarial delivery order and duplicated messages, which a correct CRDT must tolerate.

10.3 Parameters to fix before final experiments

Parameter	Role	Decision needed
L_{SLO}	latency SLO	Choose a target and justify it
$w_b,\ w_l$	objective weights	Choose, and run a sensitivity sweep
$T_{\min}, T_{\max}, S_{\min}, S_{\text{cap}}$	search box $\mathcal{X}$	Set ranges and grid resolution
λ, MMPP and Hawkes parameters	workload	Set per scenario family
$H,\ s_0,\ s_1,\ p_c$	byte model	Measure from the chosen CRDT library
$R,\ d$	network	Set per scenario family
$K_{\text{train}},\ K_{\text{test}}$	scenario split	Choose sizes; keep test traces unseen

10.4 Known limitations

- Weighted-sum solutions are sensitive to the weights, so report the Pareto front (P3) alongside any single optimum.
- The closed form is exact only under the fluid assumptions. Disagreement with the simulator under bursty load is expected and is a finding, not a bug.
- Results are conditional on the byte model. A different CRDT encoding changes κ and $\tilde{s}$, and with them the achievable saving.